

Frontend Development

Lecture 5: HTML, CSS, JavaScript

Hyunwoo Park
Graduate School of Data Science
Seoul National University
hyunwoopark@snu.ac.kr / <https://hyunwoopark.com>



Today's Lecture

The Frontend = Three Technologies:

- **HTML** - Structure (skeleton, steel)
- **CSS** - Styling (skin, wallpaper)
- **JavaScript** - Behavior (muscle, electricity)

What You'll Learn:

- How each technology works
- DOM tree and CSS box model
- CSS frameworks (Bootstrap, Tailwind)
- JavaScript evolution (vanilla → jQuery → React/Vue/Svelte)

Real-world Context: Everything users see in the browser is frontend

The Frontend Trinity

Analogy 1: Body

- HTML = skeleton (structure, bones)
- CSS = skin (appearance, color, texture)
- JS = muscle (movement, actions)

Analogy 2: House

- HTML = steel and concrete (structure, walls, rooms)
- CSS = wallpaper and paint (colors, decorations, finishes)
- JS = electricity and switches (lights, doors, interactions)

The Point: All three are essential. Remove any one and it falls apart.



Part 1

HTML - The Skeleton

What is HTML?

HTML = HyperText Markup Language

- **HyperText** = links between documents
- **Markup** = tags that describe content
- **Language** = syntax to write it

Example:

```
<h1>My Portfolio</h1>  
<p>Welcome to my fintech site</p>  
<button>Click me</button>
```

HTML's Job: Define what things ARE (structure, semantics)

HTML History: Content vs Presentation

Early HTML (1990s): Tags mixed content AND presentation

```
<font color="red" size="5">Big red text</font>  
<center>Centered text</center>  
<b>Bold text</b>
```

Problem: No semantic meaning, hard to maintain

HTML5 (2014+): Separated content from presentation

```
<header>Top of page</header>  
<article>Main content</article>  
<strong>Important text</strong>
```

Semantic tags describe MEANING, not appearance

HTML Tags & Attributes

Tags = Elements (containers for content)

```
<tagname attribute="value">Content</tagname>
```

Common tags:

- `<h1>`, `<h2>`, `<h3>` - Headings
- `<p>` - Paragraph
- `<a href="">` - Link
- `<img src="">` - Image
- `<button>` - Button
- `<div>`, `<span>` - Generic containers

HTML Nesting: Tree Structure

HTML is a tree of nested tags:

```
<body>
  <header>
    <h1>My Site</h1>
    <nav>
      <a href="/">Home</a>
      <a href="/about">About</a>
    </nav>
  </header>
</body>
```

Key: Tags must close properly (LIFO)

- Nesting matters
- Proper nesting = valid HTML

HTML Structure Example

```
<!DOCTYPE html>
<html>
  <head>
    <title>Portfolio</title>
  </head>
  <body>
    <header>
      <h1>My Portfolio</h1>
    </header>
    <main>
      <section>
        <h2>Projects</h2>
        <p>I built a dashboard</p>
      </section>
    </main>
    <footer>
      <p>&copy; 2026</p>
    </footer>
  </body>
</html>
```

Key HTML Concepts

Semantic Tags (describe meaning):

- `<header>` - top of page
- `<nav>` - navigation
- `<main>` - primary content
- `<article>` - standalone content
- `<section>` - thematic grouping
- `<footer>` - bottom of page

Form Elements:

- `<input>` - text field
- `<button>` - clickable button
- `<select>` - dropdown
- `<textarea>` - multi-line text

The Goal: HTML describes WHAT, not HOW it looks

HTML is Not Design

```
<!-- Good semantic HTML -->  
<button>Submit</button>  
  
<!-- HTML alone (no CSS)-->
```

Result: A plain, unstyled button

Point: HTML is the skeleton. Without CSS, everything looks the same (plain, boring, gray).



Part 2

CSS - The Skin

What is CSS?

CSS = Cascading Style Sheets

- **Cascading** = rules inherit and override
- **Style** = how things look (colors, sizes, spacing, layout)

Example:

```
h1 {  
  color: blue;  
  font-size: 32px;  
  margin: 20px;  
}  
  
button {  
  background-color: green;  
  padding: 10px 20px;  
  border: none;  
}
```

CSS's Job: Define how things LOOK

What Does "Sheet" Mean?

"Cascading Style Sheets"

Sheet = Collection of styling rules (like a palette)

```
/* This whole file is a "style sheet" */  
body { font-family: Arial; }  
h1 { color: blue; }  
p { font-size: 16px; }
```

Cascading = Rules flow down and override

- Global rules apply to all
- Specific rules override global
- Last rule wins

Three Ways to Style HTML

1. Inline styles (not recommended):

```
<h1 style="color: blue; font-size: 32px;">Title</h1>
```

2. Style tag (in HTML file):

```
<head>
  <style>
    h1 { color: blue; }
  </style>
</head>
```

3. External CSS file (best practice):

```
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

Bonus: SCSS (Enhanced CSS)

```
$primary-color: blue;
```

CSS3: Modern Features

CSS3 (2009+) added powerful capabilities:

Transitions & Animations:

```
button { transition: background-color 0.3s; }  
button:hover { background-color: darkblue; }
```

Transforms (scale, rotate):

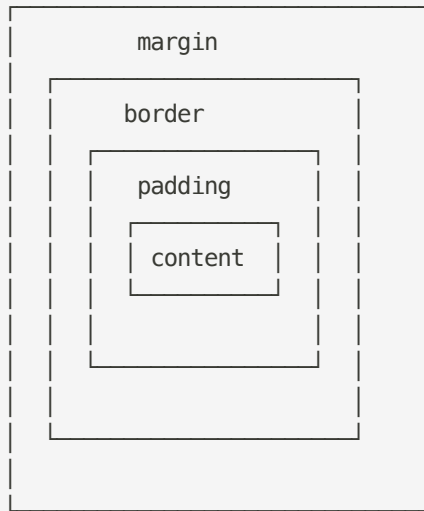
```
img:hover { transform: scale(1.1) rotate(5deg); }
```

Gradients & Shadows:

```
body { background: linear-gradient(to right, blue, purple); }  
.card { box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1); }
```


The CSS Box Model

Every HTML element is a box:



From inside out:

- Content (actual text/image)
- Padding (space inside border)
- Border (edge)

Box Model Example

```
button {  
  /* The actual text area */  
  width: 100px;  
  height: 40px;  
  
  /* Space between text and border */  
  padding: 10px;  
  
  /* Border style */  
  border: 2px solid black;  
  
  /* Space outside the button */  
  margin: 20px;  
}
```

Understanding the box model is essential for layout

CSS Selectors and Properties

Selectors (which elements):

```
button { }           /* All buttons */
.primary { }         /* Class "primary" */
#submit { }          /* ID "submit" */
.card button { }     /* Buttons inside .card */
```

Properties (what to change):

```
color: red;           /* Text color */
background-color: blue; /* Background color */
font-size: 16px;      /* Text size */
margin: 20px;         /* Space outside */
padding: 10px;        /* Space inside */
display: flex;        /* Layout system */
```

Layouts: Flex and Grid

Flexbox (1-dimensional layout):

```
.container {  
  display: flex;  
  justify-content: space-between; /* Spread items */  
  align-items: center;           /* Center vertically */  
  gap: 20px;                     /* Space between items */  
}
```

CSS Grid (2-dimensional layout):

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 2fr; /* Two columns */  
  gap: 20px;  
}
```

Modern web uses these, not tables

CSS Frameworks: Why?

Without framework: Write all CSS yourself

```
.button-primary {  
  background-color: #007bff;  
  color: white;  
  padding: 10px 20px;  
  border: none;  
  border-radius: 4px;  
  cursor: pointer;  
}
```

```
.button-secondary {  
  background-color: #6c757d;  
  color: white;  
  padding: 10px 20px;  
  border: none;  
  border-radius: 4px;  
  cursor: pointer;  
}
```

```
/* Lots of repetition! */
```

CSS Frameworks: Bootstrap

Bootstrap = Pre-built CSS library

With Bootstrap:

```
<button class="btn btn-primary">Click me</button>  
<button class="btn btn-secondary">Cancel</button>  
<button class="btn btn-success">Submit</button>
```

Bootstrap handles:

- Colors and styling
- Layout (grid system)
- Responsive design
- Components (buttons, cards, navigation)

Advantage: Fast, consistent, responsive by default

Bootstrap: In Practice

Grid system (responsive):

```
<div class="container">
  <div class="row">
    <div class="col-md-6">Half width</div>
    <div class="col-md-6">Half width</div>
  </div>
</div>
```

Pre-built components:

```
<button class="btn btn-primary btn-lg">Large</button>
<div class="card"><div class="card-body">Content</div></div>
```

CSS Frameworks: Tailwind

Tailwind = Utility-first CSS

```
<button class="bg-blue-500 text-white px-4 py-2 rounded">  
  Click me  
</button>  
  
<button class="bg-gray-600 text-white px-4 py-2 rounded">  
  Cancel  
</button>
```

Classes are atomic (single responsibility):

- `bg-blue-500` = background color
- `text-white` = text color
- `px-4` = padding left/right
- `py-2` = padding top/bottom
- `rounded` = border radius

Advantage: More control, smaller bundle, customizable

Tailwind: Building Custom Designs

Compose utilities for unique components:

```
<div class="flex items-center justify-between p-4 border rounded-lg">
  <div>
    <h3 class="text-lg font-bold">Title</h3>
    <p class="text-gray-600">Subtitle</p>
  </div>
  <button class="bg-blue-600 text-white px-4 py-2 rounded">
    Action
  </button>
</div>
```

No predefined "card" component, but flexible

Bootstrap vs Tailwind

Feature	Bootstrap	Tailwind
Components	Pre-built (button, card, nav)	Build yourself with utilities
Learning curve	Easy (grab classes)	Steeper (learn utilities)
Customization	Limited	Very customizable
Bundle size	Large	Small (unused CSS stripped)
Speed	Fast initial setup	Slower but flexible

Use Bootstrap for: Quick dashboards, MVPs, standard designs

Use Tailwind for: Custom designs, modern projects, small bundles

Design Tokens

Design Token = Named value (color, size, spacing)

```
/* Design tokens */
:root {
  --primary-color: #007bff;
  --secondary-color: #6c757d;
  --padding-small: 8px;
  --padding-large: 20px;
  --border-radius: 4px;
}

/* Use throughout */
button {
  background-color: var(--primary-color);
  padding: var(--padding-large);
  border-radius: var(--border-radius);
}
```

Why design tokens?

- **Consistency:** Same colors, sizes everywhere
- **Maintainability:** Change once, update everywhere
- **Scalability:** Easy to support dark mode, themes



Part 3

JavaScript - The Muscle

What is JavaScript?

JavaScript = Programming language for browsers

JavaScript's Job: Add interactivity (reactions, animations, data updates)

Example:

```
// When user clicks a button
button.addEventListener('click', function() {
  // Do something
  console.log('Button clicked!');
});
```

Without JS: Users can see content but can't interact

Vanilla JavaScript (The Foundation)

Vanilla JS = JavaScript without libraries

Select an element:

```
const button = document.querySelector('button');
const input = document.getElementById('username');
const items = document.querySelectorAll('.item');
```

Listen for events:

```
button.addEventListener('click', function() {
  console.log('Clicked!');
});

input.addEventListener('change', function() {
  console.log('User typed:', input.value);
});
```

Modify the page:

```
button.textContent = 'Updated';
button.style.color = 'red';
```

The DOM (Document Object Model)

DOM = JavaScript's view of the HTML

HTML:

```
<div id="app">  
  <h1>Hello</h1>  
  <button>Click</button>  
</div>
```

As a tree (DOM):

```
document  
├── html  
│   ├── head  
│   └── body  
│       ├── div#app  
│       │   ├── h1  
│       │   └── button
```

JavaScript manipulates this tree:

jQuery (The First Solution)

jQuery = Library to simplify DOM manipulation

2006 - Present (but declining)

Vanilla JS vs jQuery:

Vanilla:

```
const button = document.querySelector('button');
button.addEventListener('click', function() {
  button.style.color = 'red';
});
```

jQuery:

```
$('#button').click(function() {
  $(this).css('color', 'red');
});
```

jQuery made DOM manipulation easier, but:

- Slower than vanilla

Modern JavaScript (ES6+)

ES6 (2015) revolutionized JavaScript:

Arrow functions:

```
// Old
button.addEventListener('click', function() { });

// New
button.addEventListener('click', () => { });
```

Template literals:

```
// Old
const greeting = 'Hello ' + name;

// New
const greeting = `Hello ${name}`;
```

Destructuring:

```
const { username, email } = user;
const [first, second] = items;
```

Front-End Frameworks: Why?

Problem with Vanilla JS:

As apps grow, managing DOM becomes hard:

```
// Update user name
user.name = 'Alice';
document.getElementById('name').textContent = user.name;

// Update email
user.email = 'alice@example.com';
document.getElementById('email').textContent = user.email;

// Update avatar
user.avatar = 'alice.jpg';
document.getElementById('avatar').src = user.avatar;

// Lots of manual sync!
```

Solution: Frameworks (React, Vue, Svelte)

React

React = Library for building UIs

Concept: Components

```
function UserCard({ user }) {  
  return (  
    <div className="card">  
      <h2>{user.name}</h2>  
      <p>{user.email}</p>  
    </div>  
  );  
}
```

Reactive: When data changes, UI auto-updates

```
const [name, setName] = useState('Alice');  
  
// When user types  
const handleChange = (e) => {  
  setName(e.target.value); // Update data  
  // UI auto-updates!  
};
```

React: State & Auto-Updates

Hooks (data binding):

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <p>Count: {count}</p>  
      <button onClick={() => setCount(count + 1)}>+1</button>  
    </div>  
  );  
}
```

Auto-updates when state changes

Vue

Vue = Progressive framework

Similar to React, but simpler:

```
<template>
  <div class="card">
    <h2>{{ user.name }}</h2>
    <p>{{ user.email }}</p>
  </div>
</template>

<script>
export default {
  data() {
    return {
      user: { name: 'Alice', email: 'alice@example.com' }
    };
  }
};
</script>
```

Advantages:

- Easier to learn

Vue: Two-Way Binding

Auto-sync between input and data:

```
<template>
  <input v-model="name" />
  <p>Hello {{ name }}</p>
</template>

<script>
export default {
  data() {
    return { name: 'Alice' };
  }
};
</script>
```

Change input → updates data → updates display

Svelte

Svelte = Compiler-based framework

```
<script>
  let name = 'Alice';
</script>

<div class="card">
  <h2>{name}</h2>
  <input bind:value={name} />
</div>

<style>
  .card { padding: 20px; }
</style>
```

Advantages:

- Smallest bundle size
- Reactive by default
- Simple syntax
- Great performance

Disadvantages:

Svelte: Compiler-Based Magic

Direct variable binding (simplest):

```
<script>
  let count = 0;
</script>

<button on:click={() => count++}>
  Count: {count}
</button>

<style>
  button { padding: 10px; }
</style>
```

Simple, fast, reactive

Comparing Frontend Frameworks

Framework	Learning	Bundle	Community	Jobs	Best For
Vanilla JS	Easy	Tiny	N/A	N/A	Simple pages
jQuery	Easy	Small	Small	Few	Legacy code
React	Hard	Large	Huge	Many	Complex apps
Vue	Medium	Medium	Medium	Growing	Balanced
Svelte	Medium	Tiny	Small	Few	Modern, lean

For fintech dashboards:

- React or Vue (mature, stable)
- Might use with D3.js for charts
- Or use Streamlit (Python) instead

Framework Decision Tree

Ask yourself:

- Static content? → **Vanilla JS**
- Legacy codebase? → **jQuery** (but migrate)
- Complex app, hiring needs? → **React**
- Learning/custom designs? → **Vue**
- Performance critical? → **Svelte**

When to Use What?

Vanilla JS:

- Static pages with little interaction
- Adding interactivity to server-rendered pages
- Learning fundamentals

jQuery:

- Legacy projects (don't use for new code!)

React:

- Complex interactive apps
- Large teams
- Job requirements matter

Vue:

- Moderate complexity
- Learning frontend
- Flexibility matters

Svelte:



The Deep Dive

DOM & CSS Box Model

How Browsers Render

1. Parse HTML → Create DOM tree

HTML file → DOM tree

2. Parse CSS → Apply styling

CSS file → Match selectors → Apply rules

3. Compute layout (box model) and 4. Paint (rasterize)

Each element's size, position → Layout tree
Draw pixels on screen

5. Composite

Stack layers, handle transparency

JavaScript can modify any step!

DOM Manipulation Pattern

Before frameworks, the pattern was:

```
// 1. Get element
const input = document.querySelector('input');

// 2. Listen for change
input.addEventListener('change', (e) => {
  const value = e.target.value;

  // 3. Process data
  const result = processData(value);

  // 4. Update DOM
  document.getElementById('result').textContent = result;
});
```

Frameworks automate step 4 (update DOM)

The Problem Frameworks Solve

Manual DOM Sync (Vanilla JS):

```
user.name = 'Bob';  
document.getElementById('header-name').textContent = 'Bob';  
document.getElementById('profile-name').textContent = 'Bob';  
document.getElementById('settings-name').textContent = 'Bob';  
// Easy to forget a spot!
```

Hard to keep data and DOM in sync

Frameworks Solve This Elegantly

Declare relationship once:

```
// React/Vue/Svelte
<template>
  Name: {{ user.name }}
</template>
```

Change data, UI updates automatically:

```
user.name = 'Bob'; // All references update!
```

The insight: One source of truth (data) → multiple views (auto-synced)

CSS Box Model in Practice

```
<div class="card">
  <h2>Title</h2>
  <p>Content</p>
</div>
```

```
.card {
  width: 300px;          /* Content width */
  padding: 20px;         /* Space inside */
  border: 1px solid #ccc; /* Thin border */
  margin: 10px;          /* Space outside */
}

.card h2 {
  margin: 0;             /* Remove default margin */
  margin-bottom: 10px;   /* Add space below */
}
```

Total width on page:

```
margin(10) + border(1) + padding(20) + content(300) + padding(20) + border(1) + margin(10)
= 362px
```

Understanding this prevents layout bugs!

CSS Box Model in Practice

```
<div class="card">
  <h2>Title</h2>
  <p>Content</p>
</div>
```

```
.card {
  width: 300px;          /* Content width */
  padding: 20px;         /* Space inside */
  border: 1px solid #ccc; /* Thin border */
  margin: 10px;          /* Space outside */
}

.card h2 {
  margin: 0;             /* Remove default margin */
  margin-bottom: 10px;   /* Add space below */
}
```

Total width on page:

```
margin(10) + border(1) + padding(20) + content(300) + padding(20) + border(1) + margin(10)
= 362px
```

Understanding this prevents layout bugs!



Conclusion

Key Takeaways

The Frontend Trinity

✓ HTML - Structure

- Semantic tags describe meaning
- DOM tree is JavaScript's interface
- Never manually write layout CSS (use flexbox/grid)

✓ CSS - Styling

- Box model is fundamental
- CSS frameworks (Bootstrap/Tailwind) speed up development
- Design tokens ensure consistency

✓ JavaScript - Behavior

- Vanilla JS first, then frameworks
- React/Vue for complex apps
- Modern frameworks auto-sync data and DOM

Frontend Technology Stack

Your future stack might be:

- Frontend: React or Vue
- Styling: Tailwind (modern) or Bootstrap (quick)
- Data viz: D3.js or Plotly
- Backend: Python FastAPI
- Database: PostgreSQL

For quick dashboards: Consider Streamlit (Python) instead

For web apps: React + Tailwind + FastAPI

Resources

HTML:

- MDN Web Docs: <https://developer.mozilla.org>
- HTML5 spec: <https://html.spec.whatwg.org/>

CSS:

- Bootstrap: <https://getbootstrap.com>
- Tailwind: <https://tailwindcss.com>
- Flexbox guide: <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>

JavaScript:

- MDN JavaScript: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- Modern JS: <https://javascript.info>

Frameworks:

- React: <https://react.dev>
- Vue: <https://vuejs.org>
- Svelte: <https://svelte.dev>

Thank you.

Lecture 5: Frontend Development

Hyunwoo Park
Graduate School of Data Science
Seoul National University
hyunwoopark@snu.ac.kr / <https://hyunwoopark.com>

